

2.1沟通后细化的方案[Redis/UB虽处于两层但同机部署]

240亿IOPS（基于30-40万台Redis吞吐估算）的极高性能，设计了以下的全用户态批处理架构。

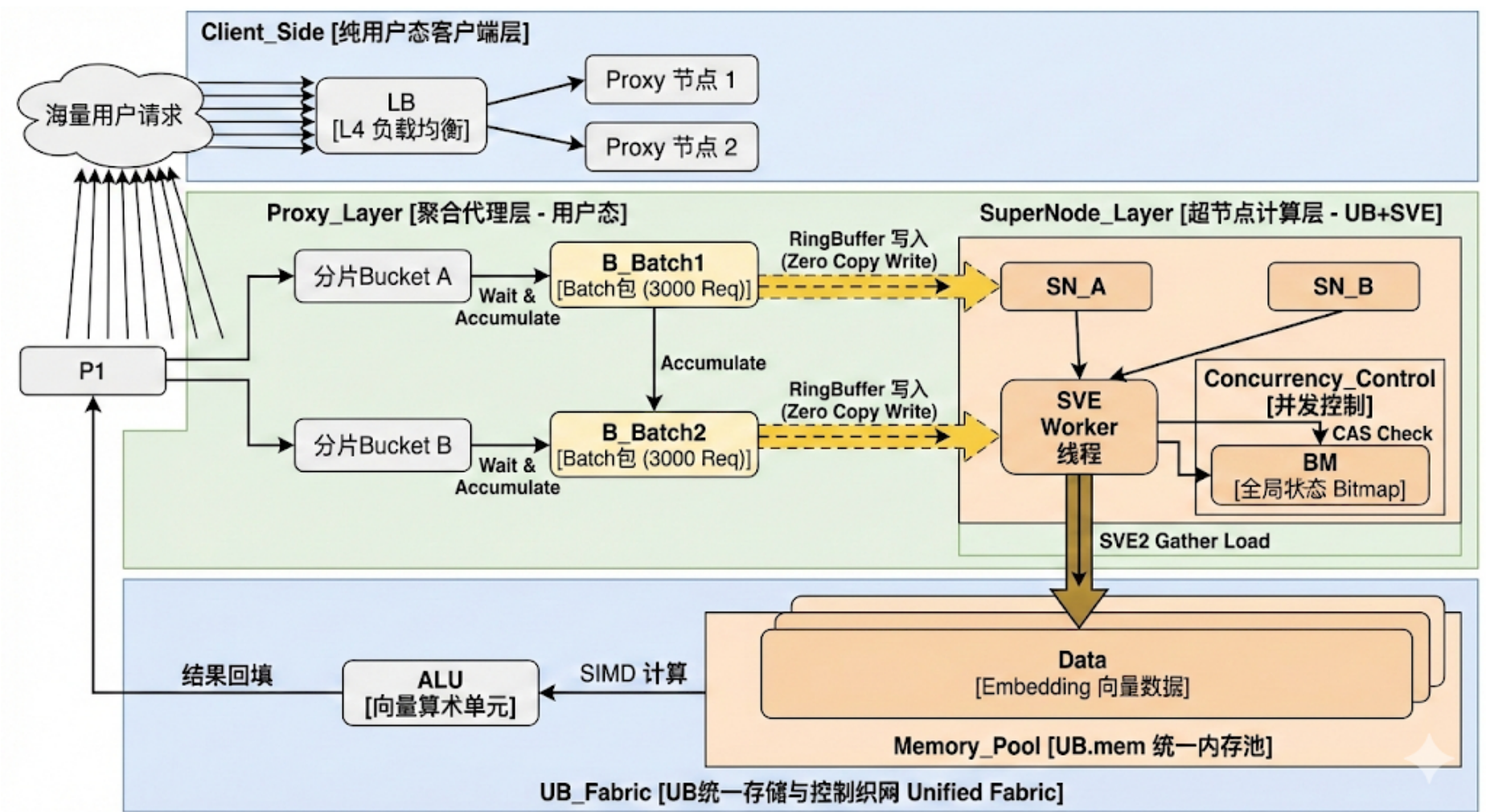
核心设计分析：关于 Bitmap + CAS 锁的必要性

在240亿 IOPS的超高吞吐下，任何锁竞争都是致命的。Bitmap CAS锁：

- 分析：如果直接对每个Embedding的读写进行互斥锁（哪怕是CAS），在读写冲突时会造成流水线停顿。但在推荐场景中，特征读取（Read）频次远高于更新（Write）（通常是 1000:1 甚至更高）。
- 结论：有必要，但需优化。不能做成传统的“阻塞等待”，而应采用无锁编程（Lock-Free）或乐观并发控制（Optimistic Concurrency Control）。
- 优化策略：使用一个全局的 StateBitmap（位于共享内存），每位对应一个特征块。
 - 写侧：原子置位（CAS 0->1），更新数据，原子复位（CAS 1->0）。
 - 读侧：检查位。如果为1（正在写），不单纯等待，而是先跳过该请求或返回旧值（Double Buffer），或者放入“重试队列”下一轮处理，保证SVE流水线不中断。

1. 总体架构设计图

主流程：Worker -> 1. SVE Gather Load (获取锁状态) -> BM Worker (内部生成 p_active) -> 2. Predicated SVE2 Gather Load (仅读取未锁项) -> Data



引入了“批量聚合层（Batch Aggregator）”，将离散的IO请求转化为批量的向量计算任务。

Code snippet

```
Code block
1 graph TD
2     subgraph Client_Side [纯用户态客户端层]
3         U[海量用户请求] -->|高并发写入| LB[LB L4 负载均衡]
4         LB -->|分发| P1[Proxy 节点 1]
```

```
5      LB -->|分发| P2[Proxy 节点 2]
6  end
7
8  subgraph Proxy_Layer [聚合代理层 - 用户态]
9      P1 -->|1.哈希分片| SB1[分片Bucket A]
10     P1 -->|1.哈希分片| SB2[分片Bucket B]
11
12     SB1 -->|2.Wait & Accumulate| B_Batch1[Batch包 (3000 Req)]
13     SB2 -->|Wait & Accumulate| B_Batch2[Batch包 (3000 Req)]
14
15     B_Batch1 -->|3.RingBuffer 写入| SN_A[超节点 A]
16     B_Batch2 -->|RingBuffer 写入| SN_B[超节点 B]
17 end
18
19 subgraph SuperNode_Layer [超节点计算层 - UB+SVE]
20     SN_A -->|4.轮询获取 Batch| Worker_A[SVE Worker 线程]
21
22     subgraph Concurrency_Control [并发控制]
23         Worker_A -->|5.CAS Check| BM[全局状态 Bitmap]
24     end
25
26     subgraph Memory_Pool [UB.mem 统一内存池]
27         BM -. ->|控制| Data[Embedding 向量数据]
28     end
29
30     Worker_A -->|6.SVE2 Gather Load| Data
31     Worker_A -->|7.SIMD 计算| ALU[向量算术单元]
32     ALU -->|8.结果回填| P1
33 end
```

2. 详细交互时序泳道图

此图展示了从请求积攒到SVE计算的全过程，特别关注“Wait”和“CAS Check”环节。

Code snippet

```
Code block
1  sequenceDiagram
2      participant User as 用户请求 (N clients)
3      participant Aggregator as 聚合代理层 (Proxy)
4      participant RB as 共享内存队列 (RingBuffer)
5      participant SVE_Worker as 超节点 SVE Worker
6      participant Bitmap as 状态位图 (CAS Lock)
7      participant UB_Mem as UB.mem (Embedding)
8
9      Note over User, Aggregator: 阶段一：积攒与分片 (Batching)
10
11     User->>Aggregator: 1. 发起并发请求 (GET uid:1..6000)
12
13     activate Aggregator
14     loop 积攒循环 (Wait Strategy)
15         Aggregator->>Aggregator: Hash(uid) -> Shard A/B
16         Aggregator->>Aggregator: 放入对应的 Bucket
17         break 当 Bucket count >= 3000 或 timeout > 500us
18         Aggregator->>Aggregator: 封包 (Batch Packet)
19     end
20 end
21
22 Aggregator->>RB: 2. 写入 Batch 包 (Zero Copy)
```

```
23 deactivate Aggregator
24
25 Note over SVE_Worker, UB_Mem: 阶段二：并发检查与向量计算
26
27 activate SVE_Worker
28 SVE_Worker->>RB: 3. 轮询获取任务
29
30 loop 遍历 Batch 中的 Key (SIMD 并行)
31     SVE_Worker->>Bitmap: 4. 预取状态位 (Prefetch)
32     SVE_Worker->>Bitmap: 5. 检查 Write_Lock (Atomic Test)
33
34     alt 状态 = 0 (无锁/空闲)
35         SVE_Worker->>UB_Mem: 6. SVE2 Gather Load (加载向量)
36         SVE_Worker->>SVE_Worker: 7. 相似度计算 (DotProduct)
37     else 状态 = 1 (正在写)
38         SVE_Worker->>SVE_Worker: 8. 标记为 Retry 或 返回默认值
39         Note right of SVE_Worker: 避免流水线停顿, 不进行 Wait
40     end
41 end
42
43 SVE_Worker-->>Aggregator: 9. 批量写回结果
44 deactivate SVE_Worker
45
46 Aggregator-->>User: 10. 拆包并返回响应
```

3. 核心代码示意 (C++ Pseudo-code)

代码分为两部分：**Proxy端的聚合逻辑** 和 **超节点端的SVE+CAS处理逻辑**。

A. Proxy端：智能聚合 (Smart Batching)

C++

```
Code block

1 // 纯用户态聚合器
2 class BatchAggregator {
3     struct BatchBucket {
4         std::vector<Request> requests;
5         int target_supernode_id;
6         uint64_t last_flush_time;
7     };
8
9     std::vector<BatchBucket> buckets; // 对应不同超节点
10    const int BATCH_LIMIT = 3000;
11    const int TIME_LIMIT_US = 200;    // 微秒级等待
12    public:
13    void EnqueueRequest(Request req) {
14        int node_id = ConsistentHash(req.key);
15        auto& bucket = buckets[node_id];
16
17        bucket.requests.push_back(req);
18
19        // 触发条件：数量达标 或 时间达标
20        if (bucket.requests.size() >= BATCH_LIMIT || IsTimeout(bucket)) {
21            FlushBatch(bucket);
22        }
23    }
24
25    void FlushBatch(BatchBucket& bucket) {
```

```

26      // 通过 UB RingBuffer 发送, 零拷贝
27      RingBuffer* rb = GetRingBuffer(bucket.target_supernode_id);
28
29      // 构造连续内存布局的 Batch 包, 利于 SVE 读取
30      BatchPacket packet = SerializeForSVE(bucket.requests);
31      rb->Push(packet);
32
33      bucket.requests.clear();
34      bucket.last_flush_time = Now();
35  }
36  };

```

B. 超节点端：CAS检查与SVE2计算 (The Core)

这是实现 **240亿 IOPS** 的关键。利用 SVE2 的 `Gather Load` 一次性读取多个地址，并利用原子指令快速检查锁状态。

C++

Code block

```

1  #include <arm_sve.h>
2  // 引入 SVE2头文件
3  // 假设 Bitmap 存在于一段共享内存中, 每个 bit 对应一个 Embedding slot
4  uint64_t* global_lock_bitmap;
5
6  void ProcessBatchSVE(BatchPacket* batch) {
7      int num_reqs = batch->count; // 例如 3000
8      uint64_t* keys = batch->keys;
9      float* results = batch->results;
10
11     // SVE 循环步长, 例如 512-bit 寄存器处理 8 个
12     uint64 keysvbool_t pg = svptrue_b64();
13
14     for (int i = 0; i < num_reqs; i += svcntd()) {
15         // 1. 加载一批 Key
16         (Offset)svuint64_t v_keys = svld1_u64(pg, &keys[i]);
17
18         // --- CAS / Bitmap Check 阶段 ---
19         // 计算 Bitmap 中的索引
20         // 假设 key 就是 slot index
21         svuint64_t v_bitmap_idx = svdiv_x(pg, v_keys, 64);
22         svuint64_t v_bit_mask = lsl_x(pg, 1UL, svmod_x(pg, v_keys, 64));
23
24         // Gather Load 对应的 Bitmap Word
25         // 注意: 这里需要支持 Gather Load 的内存地址
26         svuint64_t v_lock_states = svld1_gather_u64index(pg, global_lock_bitmap, v_bitmap_idx);
27
28         // 检测是否被锁
29         (State & Mask != 0)svuint64_t v_is_locked = svand_x(pg, v_lock_states, v_bit_mask);
30         svbool_t p_locked = svcmpne(pg, v_is_locked, 0);
31
32         // --- 数据读取阶段 ---
33         // 生成 Embedding 物理地址
34         svuint64_t v_addrs = CalculateUBAddr(v_keys);
35
36         // 关键: 对于 Locked 的项, 不仅不能读, 还要避免造成 Bus Error
37         // 使用 svnot 翻转判定条件, 只对 Unlocked 的项进行 Gather Loads
38         vbool_t p_active = svbic_b_z(pg, p_locked);
39         // Active = All - Locked
40         // SVE Gather Load (大位宽读取)
41         // 假设 Embedding 维度较小, 或者这里只演示读取第一维
42         svfloat32_t v_data = svld1_gather_u64index_f32(p_active, ub_mem_base, v_addrs);

```

```
43
44      // --- 相似度计算 (Vector Dot Product) ---
45      // 此处省略具体的向量运算逻辑...
46      // --- 异常处理 ---
47      // 对于 p_locked 为真的部分，记录到 Retry 队列或返回
48      Error Code if (svptest_any(pg, p_locked)) {
49          HandleLockedRequests(i, p_locked, batch);
50      }
51
52      // 存回结果
53      svst1_f32(p_active, &results[i], v_data);
54  }
55 }
```

4. 架构优化点总结

1. Wait & Accumulate (蓄水池策略):

- Redis层（Proxy）不仅是转发，变成了**蓄水池**。3000个请求为一个Batch，极大地减少了 RingBuffer 的交互次数（从 3000 次 push 变成 1 次）。

2. Bitmap as Optimistic Lock:

- 不使用真正的 `mutex`。使用 Bitmap 标记“正在写”。
- SVE 读取时，利用向量指令并行检查 8-16 个 Key 的锁状态。
- 关键策略**：遇到锁不等待（Wait），而是**Drop或Retry**。这保证了流水线永远满载，吞吐量最大化。

3. SVE2 Gather Load:

- 利用 `svld1_gather` 指令，一次指令周期发出多个非连续内存地址的读取请求，完美适配 UB.mem 的高并发特性。

这套架构完全运行在用户态，消除了内核切换，利用了批量化减少交互，利用了SVE2提升计算密度，需要根据硬件情况去压测确认：

- 接入层WAIT及批量转发的性能
- CAS的lock-free锁性能
- 吞吐量的瓶颈变化